Lab experiment

`

8-puzzle problem

Coding:

```
import heapq

# Goal state
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

# Manhattan Distance Heuristic
def heuristic(state):
    distance = 0

    for i in range(9):
        if state[i] != 0:
            goal_pos = goal.index(state[i])
            distance += abs(i // 3 - goal_pos // 3)
            distance += abs(i % 3 - goal_pos % 3)

    return distance

# Generate neighboring states
def get_neighbors(state):
    neighbors = []
    zero = state.index(0)

    moves = [-3, 3, -1, 1]

    for move in moves:
        new_pos = zero + move

        if 0 <= new_pos < 9:
            if move == -1 and zero % 3 == 0:
                continue
            if move == 1 and zero % 3 == 2:
                continue

            new_state = list(state)
            new_state[zero], new_state[new_pos] = \
```

```
                new_state[new_pos], new_state[zero]

            neighbors.append(tuple(new_state))

    return neighbors

# Check whether puzzle is solvable
def is_solvable(state):
    nums = [x for x in state if x != 0]
    inversions = 0

    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] > nums[j]:
                inversions += 1

    return inversions % 2 == 0

# A* Search Algorithm
def solve(start):
    if not is_solvable(start):
        print("Puzzle is not solvable")
        return

    pq = []
    heapq.heappush(pq, (heuristic(start), 0, start))

    parent = {start: None}
    cost = {start: 0}

    while pq:
        f, g, current = heapq.heappop(pq)

        if current == goal:
            path = []

            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()

            print("Solution found!")
            print("Number of moves:", len(path) - 1)
```

```
            for state in path:
                for i in range(0, 9, 3):
                    print(state[i:i+3])
                print()

            return

        for neighbor in get_neighbors(current):
            new_cost = g + 1

            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                parent[neighbor] = current

                f = new_cost + heuristic(neighbor)

                heapq.heappush(
                    pq, (f, new_cost, neighbor)
                )

    print("No solution found")

# Initial state
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

solve(start)
```

Output:

Output

```
Solution found!
Number of moves: 2
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```